Chapter 18. SQL and Big Data

While most of the content in this book covers the various features of the SQL language when using a relational database such as MySQL, the data landscape has changed quite a bit over the past decade, and SQL is changing to meet the needs of today's rapidly evolving environments. Many organizations that had used relational databases exclusively just a few years ago are now also housing data in Hadoop clusters, data lakes, and NoSQL databases. At the same time, companies are struggling to find ways to gain insights from the ever-growing volumes of data, and the fact that this data is now spread across multiple data stores, perhaps both on-site and in the cloud, makes this a daunting task.

Because SQL is used by millions of people and has been integrated into thousands of applications, it makes sense to leverage SQL to harness this data and make it actionable. Over the past several years, a new breed of tools has emerged to enable SQL access to structured, semi-structured, and unstructured data: tools such as Presto, Apache Drill, and Toad Data Point. This chapter explores one of these tools, Apache Drill, to demonstrate how data in different formats and stored on different servers can be brought together for reporting and analysis.

Introduction to Apache Drill

There have been numerous tools and interfaces developed to allow SQL access to data stored in Hadoop, NoSQL, Spark, and cloud-based distributed filesystems. Examples include Hive, which was one of the first attempts to allow users to query data stored in Hadoop, and Spark SQL, which is a library used to query data stored in various formats from within Spark. One relative newcomer is the open source Apache Drill, which first hit the scene in 2015 and has the following compelling features:

- Facilitates queries across multiple data formats, including delimited data, JSON, Parquet, and log files
- Connects to relational databases, Hadoop, NoSQL, HBase, and Kafka, as well as specialized data formats such as PCAP, BlockChain, and others
- Allows creation of custom plug-ins to connect to most any other data store

- Requires no up-front schema definitions
- Supports the SQL:2003 standard
- Works with popular business intelligence (BI) tools like Tableau and Apache Superset

Using Drill, you can connect to any number of data sources and begin querying, without the need to first set up a metadata repository. While it is beyond the scope of this book to discuss the installation and configuration options for Apache Drill, if you are interested in learning more, I highly recommend *Learning Apache Drill* by Charles Givre and Paul Rogers (O'Reilly).

Querying Files Using Drill

Let's start by using Drill to query data in a file. Drill understands how to read several different file formats, including packet capture (PCAP) files, which are in binary format and contain information about packets traveling over a network. All I have to do when I want to query a PCAP file is to configure Drill's `dfs` (distributed filesystem) plug-in to include the path to the directory containing my files, and I'm ready to write queries.

The first thing I'd like to do is find out what columns are available in the file I will be querying. Drill includes partial support for `information_schema` (covered in Chapter 15), so you can find out high-level information about the data files in your workspace:

```
apache drill> SELECT file_name, is_directory, is_file, permission
. . . . . > FROM information_schema.`files`
. . . . . > WHERE schema_name = 'dfs.data';
+------------------+-------------+---------+------------+
|    file_name     | is_directory | is_file | permission |
+------------------+-------------+---------+------------+
| attack-trace.pcap | false       | true    | rwxrwx---  |
+------------------+-------------+---------+------------+
1 row selected (0.238 seconds)
```

The results show that I have a single file named *attack-trace.pcap* in my data workspace, which is useful information, but I can't query `information_schema.columns` to find out what columns are available in the file. However, executing a query that returns no rows against the file will show the set of available columns:[1]

```
apache drill> SELECT * FROM dfs.data.`attack-trace.pcap`
. . . . . > WHERE 1=2;
+------+---------+-----------+-----------------+--------+--------+
| type | network | timestamp | timestamp_micro | src_ip | dst_ip |
+------+---------+-----------+-----------------+--------+--------+
  ----------+----------+-----------------+-----------------+-------------+
   src_port | dst_port | src_mac_address | dst_mac_address | tcp_session |
  ----------+----------+-----------------+-----------------+-------------+
  ---------+-----------+-------------+--------------+---------------+
   tcp_ack | tcp_flags | tcp_flags_ns | tcp_flags_cwr | tcp_flags_ece  |
  ---------+-----------+-------------+--------------+---------------+
  -------------------------+-----------------------------------+
   tcp_flags_ece_ecn_capable | tcp_flags_ece_congestion_experienced |
  -------------------------+-----------------------------------+
  --------------+--------------+--------------+--------------+
   tcp_flags_urg | tcp_flags_ack | tcp_flags_psh | tcp_flags_rst |
  --------------+--------------+--------------+--------------+
  --------------+--------------+-----------------+--------------+
   tcp_flags_syn | tcp_flags_fin | tcp_parsed_flags | packet_length |
  --------------+--------------+-----------------+--------------+
  -----------+------+
   is_corrupt | data |
  -----------+------+

No rows selected (0.285 seconds)
```

Now that I know the names of the columns in a PCAP file, I'm ready to write queries. Here's a query that counts the number of packets sent from each IP address to each destination port:

```
apache drill> SELECT src_ip, dst_port,
. . . . . >   count(*) AS packet_count
. . . . . > FROM dfs.data.`attack-trace.pcap`
. . . . . > GROUP BY src_ip, dst_port;
+---------------+----------+--------------+
|     src_ip    | dst_port | packet_count |
+---------------+----------+--------------+
| 98.114.205.102 | 445      | 18           |
| 192.150.11.111 | 1821     | 3            |
| 192.150.11.111 | 1828     | 17           |
| 98.114.205.102 | 1957     | 6            |
| 192.150.11.111 | 1924     | 6            |
| 192.150.11.111 | 8884     | 15           |
| 98.114.205.102 | 36296    | 12           |
```

```
| 98.114.205.102 | 1080     | 159         |
| 192.150.11.111 | 2152     | 112         |
+----------------+----------+-------------+
9 rows selected (0.254 seconds)
```

Here's another query that aggregates packet information for each second:

```
apache drill> SELECT trunc(extract(second from `timestamp`)) as packet_time,
. . . . . >   count(*) AS num_packets,
. . . . . >   sum(packet_length) AS tot_volume
. . . . . > FROM dfs.data.`attack-trace.pcap`
. . . . . > GROUP BY trunc(extract(second from `timestamp`));
+-------------+-------------+------------+
| packet_time | num_packets | tot_volume |
+-------------+-------------+------------+
| 28.0        | 15          | 1260       |
| 29.0        | 12          | 1809       |
| 30.0        | 13          | 4292       |
| 31.0        | 3           | 286        |
| 32.0        | 2           | 118        |
| 33.0        | 15          | 1054       |
| 34.0        | 35          | 14446      |
| 35.0        | 29          | 16926      |
| 36.0        | 25          | 16710      |
| 37.0        | 25          | 16710      |
| 38.0        | 26          | 17788      |
| 39.0        | 23          | 15578      |
| 40.0        | 25          | 16710      |
| 41.0        | 23          | 15578      |
| 42.0        | 30          | 20052      |
| 43.0        | 25          | 16710      |
| 44.0        | 22          | 7484       |
+-------------+-------------+------------+
17 rows selected (0.422 seconds)
```

In this query, I needed to put backticks (`) around `timestamp` because it is a reserved word.

You can query files stored locally, on your network, in a distributed filesystem, or in the cloud. Drill has built-in support for many file types, but you can also build your own plug-in to allow Drill to query any type of file. The next two sections will explore querying data stored in a database.

Querying MySQL Using Drill

Drill can connect to any relational database via a JDBC driver, so the next logical step is to show how Drill can query the Sakila sample database used for the examples in this book. All you need to do to get started is to load the JDBC driver for MySQL and configure Drill to connect to the MySQL database.

NOTE

At this point, you may be wondering, "Why would I use Drill to query MySQL?" One reason is that (as you will see at the end of this chapter) you can write queries using Drill that combine data from different sources, so you might write a query that joins data from MySQL, Hadoop, and comma-delimited files, for example.

The first step is to choose a database:

```
apache drill (information_schema)> use mysql.sakila;
+------+----------------------------------------+
|  ok  |              summary                   |
+------+----------------------------------------+
| true | Default schema changed to [mysql.sakila] |
+------+----------------------------------------+
1 row selected (0.062 seconds)
```

After choosing the database, you can issue the `show tables` command to see all of the tables available in the chosen schema:

```
apache drill (mysql.sakila)> show tables;
+--------------+----------------------------+
| TABLE_SCHEMA |        TABLE_NAME          |
+--------------+----------------------------+
| mysql.sakila | actor                      |
| mysql.sakila | address                    |
| mysql.sakila | category                   |
| mysql.sakila | city                       |
| mysql.sakila | country                    |
| mysql.sakila | customer                   |
| mysql.sakila | film                       |
| mysql.sakila | film_actor                 |
| mysql.sakila | film_category              |
```

```
| mysql.sakila | film_text                |
| mysql.sakila | inventory                |
| mysql.sakila | language                 |
| mysql.sakila | payment                  |
| mysql.sakila | rental                   |
| mysql.sakila | sales                    |
| mysql.sakila | staff                    |
| mysql.sakila | store                    |
| mysql.sakila | actor_info               |
| mysql.sakila | customer_list            |
| mysql.sakila | film_list                |
| mysql.sakila | nicer_but_slower_film_list |
| mysql.sakila | sales_by_film_category   |
| mysql.sakila | sales_by_store           |
| mysql.sakila | staff_list               |
+--------------+--------------------------+
24 rows selected (0.147 seconds)
```

I will start by executing a few queries demonstrated in earlier chapters. Here's a simple two-table join from Chapter 5:

```
apache drill (mysql.sakila)> SELECT a.address_id, a.address, ct.city
. . . . . )> FROM address a
. . . . . )>   INNER JOIN city ct
. . . . . )>   ON a.city_id = ct.city_id
. . . . . )> WHERE a.district = 'California';
+------------+----------------------+---------------+
| address_id |       address        |     city      |
+------------+----------------------+---------------+
| 6          | 1121 Loja Avenue     | San Bernardino |
| 18         | 770 Bydgoszcz Avenue | Citrus Heights |
| 55         | 1135 Izumisano Parkway | Fontana      |
| 116        | 793 Cam Ranh Avenue  | Lancaster     |
| 186        | 533 al-Ayn Boulevard | Compton       |
| 218        | 226 Brest Manor      | Sunnyvale     |
| 274        | 920 Kumbakonam Loop  | Salinas       |
| 425        | 1866 al-Qatif Avenue | El Monte      |
| 599        | 1895 Zhezqazghan Drive | Garden Grove |
+------------+----------------------+---------------+
9 rows selected (3.523 seconds)
```

The next query comes from Chapter 8 and includes both a `group by` clause and a `having` clause:

```
apache drill (mysql.sakila)> SELECT fa.actor_id, f.rating,
. . . . . )>   count(*) num_films
. . . . . )> FROM film_actor fa
. . . . . )>   INNER JOIN film f
. . . . . )>   ON fa.film_id = f.film_id
. . . . . )> WHERE f.rating IN ('G','PG')
. . . . . )> GROUP BY fa.actor_id, f.rating
. . . . . )> HAVING count(*) > 9;
+----------+--------+-----------+
| actor_id | rating | num_films |
+----------+--------+-----------+
| 137      | PG     | 10        |
| 37       | PG     | 12        |
| 180      | PG     | 12        |
| 7        | G      | 10        |
| 83       | G      | 14        |
| 129      | G      | 12        |
| 111      | PG     | 15        |
| 44       | PG     | 12        |
| 26       | PG     | 11        |
| 92       | PG     | 12        |
| 17       | G      | 12        |
| 158      | PG     | 10        |
| 147      | PG     | 10        |
| 14       | G      | 10        |
| 102      | PG     | 11        |
| 133      | PG     | 10        |
+----------+--------+-----------+
16 rows selected (0.277 seconds)
```

Finally, here is a query from Chapter 16 that includes three different ranking functions:

```
apache drill (mysql.sakila)> SELECT customer_id, count(*) num_rentals,
. . . . . )>   row_number()
. . . . . )>     over (order by count(*) desc)
. . . . . )>       row_number_rnk,
. . . . . )>   rank()
. . . . . )>     over (order by count(*) desc) rank_rnk,
. . . . . )>   dense_rank()
. . . . . )>     over (order by count(*) desc)
. . . . . )>       dense_rank_rnk
. . . . . )> FROM rental
. . . . . )> GROUP BY customer_id
. . . . . )> ORDER BY 2 desc;
```

```
+-------------+-------------+---------------+----------+---------------+
| customer_id | num_rentals | row_number_rnk | rank_rnk | dense_rank_rnk |
+-------------+-------------+---------------+----------+---------------+
| 148         | 46          | 1             | 1        | 1             |
| 526         | 45          | 2             | 2        | 2             |
| 144         | 42          | 3             | 3        | 3             |
| 236         | 42          | 4             | 3        | 3             |
| 75          | 41          | 5             | 5        | 4             |
| 197         | 40          | 6             | 6        | 5             |
...
| 248         | 15          | 595           | 594      | 30            |
| 61          | 14          | 596           | 596      | 31            |
| 110         | 14          | 597           | 596      | 31            |
| 281         | 14          | 598           | 596      | 31            |
| 318         | 12          | 599           | 599      | 32            |
+-------------+-------------+---------------+----------+---------------+
599 rows selected (1.827 seconds)
```

These few examples demonstrate Drill's ability to execute reasonably complex queries against MySQL, but you will need to keep in mind that Drill works with many relational databases, not just MySQL, so some features of the language may differ (e.g., data conversion functions). For more information, read Drill's documentation about their SQL implementation.

Querying MongoDB Using Drill

After using Drill to query the sample Sakila data in MySQL, the next logical step is to convert the Sakila data to another commonly used format, store it in a nonrelational database, and use Drill to query the data. I decided to convert the data to JSON and store it in MongoDB, which is one of the more popular NoSQL platforms for document storage. Drill includes a plug-in for MongoDB and also understands how to read JSON documents, so it was relatively easy to load the JSON files into Mongo and begin writing queries.

Before diving into the queries, let's take a look at the structure of the JSON files, since they aren't in normalized form. The first of the two JSON files is *films.json*:

```
{"_id":1,
 "Actors":[
   {"First name":"PENELOPE","Last name":"GUINESS","actorId":1},
   {"First name":"CHRISTIAN","Last name":"GABLE","actorId":10},
```

```
  {"First name":"LUCILLE","Last name":"TRACY","actorId":20},
  {"First name":"SANDRA","Last name":"PECK","actorId":30},
  {"First name":"JOHNNY","Last name":"CAGE","actorId":40},
  {"First name":"MENA","Last name":"TEMPLE","actorId":53},
  {"First name":"WARREN","Last name":"NOLTE","actorId":108},
  {"First name":"OPRAH","Last name":"KILMER","actorId":162},
  {"First name":"ROCK","Last name":"DUKAKIS","actorId":188},
  {"First name":"MARY","Last name":"KEITEL","actorId":198}],
 "Category":"Documentary",
 "Description":"A Epic Drama of a Feminist And a Mad Scientist
   who must Battle a Teacher in The Canadian Rockies",
 "Length":"86",
 "Rating":"PG",
 "Rental Duration":"6",
 "Replacement Cost":"20.99",
 "Special Features":"Deleted Scenes,Behind the Scenes",
 "Title":"ACADEMY DINOSAUR"},
{"_id":2,
 "Actors":[
  {"First name":"BOB","Last name":"FAWCETT","actorId":19},
  {"First name":"MINNIE","Last name":"ZELLWEGER","actorId":85},
  {"First name":"SEAN","Last name":"GUINESS","actorId":90},
  {"First name":"CHRIS","Last name":"DEPP","actorId":160}],
 "Category":"Horror",
 "Description":"A Astounding Epistle of a Database Administrator
   And a Explorer who must Find a Car in Ancient China",
 "Length":"48",
 "Rating":"G",
 "Rental Duration":"3",
 "Replacement Cost":"12.99",
 "Special Features":"Trailers,Deleted Scenes",
 "Title":"ACE GOLDFINGER"},
...
{"_id":999,
 "Actors":[
  {"First name":"CARMEN","Last name":"HUNT","actorId":52},
  {"First name":"MARY","Last name":"TANDY","actorId":66},
  {"First name":"PENELOPE","Last name":"CRONYN","actorId":104},
  {"First name":"WHOOPI","Last name":"HURT","actorId":140},
  {"First name":"JADA","Last name":"RYDER","actorId":142}],
 "Category":"Children",
 "Description":"A Fateful Reflection of a Waitress And a Boat
   who must Discover a Sumo Wrestler in Ancient China",
 "Length":"101",
 "Rating":"R",
 "Rental Duration":"5",
```

```
 "Replacement Cost":"28.99",
 "Special Features":"Trailers,Deleted Scenes",
 "Title":"ZOOLANDER FICTION"}
{"_id":1000,
 "Actors":[
  {"First name":"IAN","Last name":"TANDY","actorId":155},
  {"First name":"NICK","Last name":"DEGENERES","actorId":166},
  {"First name":"LISA","Last name":"MONROE","actorId":178}],
 "Category":"Comedy",
 "Description":"A Intrepid Panorama of a Mad Scientist And a Boy
   who must Redeem a Boy in A Monastery",
 "Length":"50",
 "Rating":"NC-17",
 "Rental Duration":"3",
 "Replacement Cost":"18.99",
 "Special Features":
 "Trailers,Commentaries,Behind the Scenes",
 "Title":"ZORRO ARK"}
```


There are 1,000 documents in this collection, and each document contains a number of scalar attributes (`Title` , `Rating` , `_id`) but also includes a list called `Actors` , which contains 1 to *N* elements consisting of the actor ID, first name, and last name attributes for every actor appearing in the film. Therefore, this file contains all of the data found in the `actor` , `film` , and `film_actor` tables within the MySQL Sakila database.

The second file is *customer.json*, which combines data from the `customer` , `address` , `city` , `country` , `rental` , and `payment` tables from the MySQL Sakila database:


```
{"_id":1,
 "Address":"1913 Hanoi Way",
 "City":"Sasebo",
 "Country":"Japan",
 "District":"Nagasaki",
 "First Name":"MARY",
 "Last Name":"SMITH",
 "Phone":"28303384290",
 "Rentals":[
  {"rentalId":1185,
   "filmId":611,
   "staffId":2,
   "Film Title":"MUSKETEERS WAIT",
   "Payments":[
    {"Payment Id":3,"Amount":5.99,"Payment Date":"2005-06-15 00:54:12"}],
   "Rental Date":"2005-06-15 00:54:12.0",
```

```
    "Return Date":"2005-06-23 02:42:12.0"},
   {"rentalId":1476,
    "filmId":308,
    "staffId":1,
    "Film Title":"FERRIS MOTHER",
    "Payments":[
     {"Payment Id":5,"Amount":9.99,"Payment Date":"2005-06-15 21:08:46"}],
    "Rental Date":"2005-06-15 21:08:46.0",
    "Return Date":"2005-06-25 02:26:46.0"},
...
   {"rentalId":14825,
    "filmId":317,
    "staffId":2,
    "Film Title":"FIREBALL PHILADELPHIA",
    "Payments":[
     {"Payment Id":30,"Amount":1.99,"Payment Date":"2005-08-22 01:27:57"}],
    "Rental Date":"2005-08-22 01:27:57.0",
    "Return Date":"2005-08-27 07:01:57.0"}
  ]
}
```


This file contains 599 entries (only one was shown here), which are loaded into Mongo as 599 documents in the `customers` collection. Each document contains the information about a single customer, along with all of the rentals and associated payments made by that customer. Furthermore, the documents contain nested lists, since each rental in the `Rentals` list also contains a list of `Payments`.

After the JSON files have been loaded, the Mongo database contains two collections (`films` and `customers`), and the data in these collections spans nine different tables from the MySQL Sakila database. This is a fairly typical scenario, since application programmers typically work with collections and generally prefer not to deconstruct their data for storage into normalized relational tables. The challenge from an SQL perspective is to determine how to flatten this data so that it behaves as if it were stored in multiple tables.

To illustrate, let's construct the following query against the `films` collection: find all actors who have appeared in 10 or more films rated either G or PG. Here's what the raw data looks like:

```
apache drill (mongo.sakila)> SELECT Rating, Actors
. . . . . .)> FROM films
. . . . . .)> WHERE Rating IN ('G','PG');
```

```
+--------+-------------------------------------------------------------+
| Rating |                            Actors                           |
+--------+-------------------------------------------------------------+
| PG     |[{"First name":"PENELOPE","Last name":"GUINESS","actorId":"1"},
         {"First name":"FRANCES","Last name":"DAY-LEWIS","actorId":"48"},
         {"First name":"ANNE","Last name":"CRONYN","actorId":"49"},
         {"First name":"RAY","Last name":"JOHANSSON","actorId":"64"},
         {"First name":"PENELOPE","Last name":"CRONYN","actorId":"104"},
         {"First name":"HARRISON","Last name":"BALE","actorId":"115"},
         {"First name":"JEFF","Last name":"SILVERSTONE","actorId":"180"},
         {"First name":"ROCK","Last name":"DUKAKIS","actorId":"188"}] |
| PG     |[{"First name":"UMA","Last name":"WOOD","actorId":"13"},
         {"First name":"HELEN","Last name":"VOIGHT","actorId":"17"},
         {"First name":"CAMERON","Last name":"STREEP","actorId":"24"},
         {"First name":"CARMEN","Last name":"HUNT","actorId":"52"},
         {"First name":"JANE","Last name":"JACKMAN","actorId":"131"},
         {"First name":"BELA","Last name":"WALKEN","actorId":"196"}] |
...
| G      |[{"First name":"ED","Last name":"CHASE","actorId":"3"},
         {"First name":"JULIA","Last name":"MCQUEEN","actorId":"27"},
         {"First name":"JAMES","Last name":"PITT","actorId":"84"},
         {"First name":"CHRISTOPHER","Last name":"WEST","actorId":"163"},
         {"First name":"MENA","Last name":"HOPPER","actorId":"170"}] |
+--------+-------------------------------------------------------------+
372 rows selected (0.432 seconds)
```

The `Actors` field is a list of one or more actor documents. In order to interact with this data as if it were a table, the `flatten` command can be used to turn the list into a nested table containing three fields:

```
apache drill (mongo.sakila)> SELECT f.Rating, flatten(Actors) actor_list
. . . . . .)>   FROM films f
. . . . . .)>   WHERE f.Rating IN ('G','PG');
+--------+-------------------------------------------------------------+
| Rating |                          actor_list                         |
+--------+-------------------------------------------------------------+
| PG     | {"First name":"PENELOPE","Last name":"GUINESS","actorId":"1"}  |
| PG     | {"First name":"FRANCES","Last name":"DAY-LEWIS","actorId":"48"}|
| PG     | {"First name":"ANNE","Last name":"CRONYN","actorId":"49"}      |
| PG     | {"First name":"RAY","Last name":"JOHANSSON","actorId":"64"}    |
| PG     | {"First name":"PENELOPE","Last name":"CRONYN","actorId":"104"} |
| PG     | {"First name":"HARRISON","Last name":"BALE","actorId":"115"}   |
| PG     | {"First name":"JEFF","Last name":"SILVERSTONE","actorId":"180"}|
| PG     | {"First name":"ROCK","Last name":"DUKAKIS","actorId":"188"}    |
```

```
| PG    | {"First name":"UMA","Last name":"WOOD","actorId":"13"}        |
| PG    | {"First name":"HELEN","Last name":"VOIGHT","actorId":"17"}    |
| PG    | {"First name":"CAMERON","Last name":"STREEP","actorId":"24"}  |
| PG    | {"First name":"CARMEN","Last name":"HUNT","actorId":"52"}     |
| PG    | {"First name":"JANE","Last name":"JACKMAN","actorId":"131"}   |
| PG    | {"First name":"BELA","Last name":"WALKEN","actorId":"196"}    |
...
| G     | {"First name":"ED","Last name":"CHASE","actorId":"3"}         |
| G     | {"First name":"JULIA","Last name":"MCQUEEN","actorId":"27"}   |
| G     | {"First name":"JAMES","Last name":"PITT","actorId":"84"}      |
| G     | {"First name":"CHRISTOPHER","Last name":"WEST","actorId":"163"}|
| G     | {"First name":"MENA","Last name":"HOPPER","actorId":"170"}    |
+--------+-------------------------------------------------------------+
2,119 rows selected (0.718 seconds)     |
```

This query returns 2,119 rows, rather than the 372 rows returned by the previous query, which indicates that on average 5.7 actors appear in each G or PG film. This query can then be wrapped in a subquery and used to group the data by rating and actor, as in:

```
apache drill (mongo.sakila)> SELECT g_pg_films.Rating,
. . . . . )>   g_pg_films.actor_list.`First name` first_name,
. . . . . )>   g_pg_films.actor_list.`Last name` last_name,
. . . . . )>   count(*) num_films
. . . . . )> FROM
. . . . . )>  (SELECT f.Rating, flatten(Actors) actor_list
. . . . . )>   FROM films f
. . . . . )>   WHERE f.Rating IN ('G','PG')
. . . . . )>  ) g_pg_films
. . . . . )> GROUP BY g_pg_films.Rating,
. . . . . )>   g_pg_films.actor_list.`First name`,
. . . . . )>   g_pg_films.actor_list.`Last name`
. . . . . )> HAVING count(*) > 9;
+--------+-----------+------------+-----------+
| Rating | first_name |  last_name  | num_films |
+--------+-----------+------------+-----------+
| PG    | JEFF      | SILVERSTONE | 12       |
| G     | GRACE     | MOSTEL     | 10       |
| PG    | WALTER    | TORN       | 11       |
| PG    | SUSAN     | DAVIS      | 10       |
| PG    | CAMERON   | ZELLWEGER  | 15       |
| PG    | RIP       | CRAWFORD   | 11       |
| PG    | RICHARD   | PENN       | 10       |
| G     | SUSAN     | DAVIS      | 13       |
| PG    | VAL       | BOLGER     | 12       |
```

```
| PG    | KIRSTEN    | AKROYD     | 12       |
| G     | VIVIEN     | BERGEN     | 10       |
| G     | BEN        | WILLIS     | 14       |
| G     | HELEN      | VOIGHT     | 12       |
| PG    | VIVIEN     | BASINGER   | 10       |
| PG    | NICK       | STALLONE   | 12       |
| G     | DARYL      | CRAWFORD   | 12       |
| PG    | MORGAN     | WILLIAMS   | 10       |
| PG    | FAY        | WINSLET    | 10       |
+--------+------------+-------------+-----------+
18 rows selected (0.466 seconds)
```

The inner query uses the `flatten` command to create one row for every actor who has appeared in a G or PG movie, and the outer query simply performs a grouping on this data set.

Next, let's write a query against the `customers` collection in Mongo. This is a bit more challenging since each document contains a list of film rentals, each of which contains a list of payments. To make it a little more interesting, let's also join to the `films` collection in order to see how Drill handles joins. The query should return all customers who have spent more than $80 to rent films rated either G or PG. Here's what it looks like:

```
apache drill (mongo.sakila)> SELECT first_name, last_name,
. . . . . )>   sum(cast(cust_payments.payment_data.Amount
. . . . . )>       as decimal(4,2))) tot_payments
. . . . . )> FROM
. . . . . )>  (SELECT cust_data.first_name,
. . . . . )>    cust_data.last_name,
. . . . . )>    f.Rating,
. . . . . )>    flatten(cust_data.rental_data.Payments)
. . . . . )>     payment_data
. . . . . )>   FROM films f
. . . . . )>    INNER JOIN
. . . . . )>   (SELECT c.`First Name` first_name,
. . . . . )>     c.`Last Name` last_name,
. . . . . )>     flatten(c.Rentals) rental_data
. . . . . )>    FROM customers c
. . . . . )>   ) cust_data
. . . . . )>    ON f._id = cust_data.rental_data.filmID
. . . . . )>   WHERE f.Rating IN ('G','PG')
. . . . . )>  ) cust_payments
. . . . . )> GROUP BY first_name, last_name
. . . . . )> HAVING
. . . . . )>   sum(cast(cust_payments.payment_data.Amount
```

```
. . . . . )>       as decimal(4,2))) > 80;
+------------+-----------+--------------+
| first_name | last_name | tot_payments |
+------------+-----------+--------------+
| ELEANOR    | HUNT      | 85.80        |
| GORDON     | ALLARD    | 85.86        |
| CLARA      | SHAW      | 86.83        |
| JACQUELINE | LONG      | 86.82        |
| KARL       | SEAL      | 89.83        |
| PRISCILLA  | LOWE      | 95.80        |
| MONICA     | HICKS     | 85.82        |
| LOUIS      | LEONE     | 95.82        |
| JUNE       | CARROLL   | 88.83        |
| ALICE      | STEWART   | 81.82        |
+------------+-----------+--------------+
10 rows selected (1.658 seconds)
```

The innermost query, which I named `cust_data`, flattens the `Rentals` list so that the `cust_payments` query can join to the `films` collection and also flatten the `Payments` list. The outermost query groups the data by customer name and applies a `having` clause to filter out customers who spent $80 or less on films rated G or PG.

Drill with Multiple Data Sources

So far, I have used Drill to join multiple tables stored in the same database, but what if the data is stored in different databases? For example, let's say the customer/rental/payment data is stored in MongoDB but the catalog of film/actor data is stored in MySQL. As long as Drill is configured to connect to both databases, you just need to describe where to find the data. Here's the query from the previous section, but instead of joining to the films collection stored in MongoDB, the join specifies the film table stored in MySQL:

```
apache drill (mongo.sakila)> SELECT first_name, last_name,
. . . . . )>   sum(cast(cust_payments.payment_data.Amount
. . . . . )>       as decimal(4,2))) tot_payments
. . . . . )> FROM
. . . . . )>  (SELECT cust_data.first_name,
. . . . . )>    cust_data.last_name,
. . . . . )>    f.Rating,
. . . . . )>    flatten(cust_data.rental_data.Payments)
. . . . . )>      payment_data
. . . . . )>   FROM mysql.sakila.film f
```

```
. . . . . )>    INNER JOIN
. . . . . )>   (SELECT c.`First Name` first_name,
. . . . . )>     c.`Last Name` last_name,
. . . . . )>     flatten(c.Rentals) rental_data
. . . . . )>    FROM mongo.sakila.customers c
. . . . . )>   ) cust_data
. . . . . )>    ON f.film_id =
. . . . . )>     cast(cust_data.rental_data.filmID as integer)
. . . . . )>  WHERE f.rating IN ('G','PG')
. . . . . )> ) cust_payments
. . . . . )> GROUP BY first_name, last_name
. . . . . )> HAVING
. . . . . )>  sum(cast(cust_payments.payment_data.Amount
. . . . . )>       as decimal(4,2))) > 80;
+------------+-----------+--------------+
| first_name | last_name | tot_payments |
+------------+-----------+--------------+
| LOUIS      | LEONE     | 95.82        |
| JACQUELINE | LONG      | 86.82        |
| CLARA      | SHAW      | 86.83        |
| ELEANOR    | HUNT      | 85.80        |
| JUNE       | CARROLL   | 88.83        |
| PRISCILLA  | LOWE      | 95.80        |
| ALICE      | STEWART   | 81.82        |
| MONICA     | HICKS     | 85.82        |
| GORDON     | ALLARD    | 85.86        |
| KARL       | SEAL      | 89.83        |
+------------+-----------+--------------+
10 rows selected (1.874 seconds)
```

Since I'm using multiple databases in the same query, I specified the full path to each table/collection to make it clear as to where the data is being sourced. This is where Drill really shines, since I can combine data from multiple sources in the same query without having to transform and load the data from one source to another.

Future of SQL

The future of relational databases is somewhat unclear. It is possible that the big data technologies of the past decade will continue to mature and gain market share. It's also possible that a new set of technologies will emerge, overtaking Hadoop and NoSQL, and taking additional market share from relational databases. However, most companies still

run their core business functions using relational databases, and it should take a long time for this to change.

The future of SQL seems a bit clearer, however. While the SQL language started out as a mechanism for interacting with data in relational databases, tools like Apache Drill act more like an abstraction layer, facilitating the analysis of data across various database platforms. In this author's opinion, this trend will continue, and SQL will remain a critical tool for data analysis and reporting for many years.

1 These results show the columns in the file based on Drill's understanding of the PCAP file structure. If you query a file whose format is not known to Drill, the result set will contain an array of strings with a single column named `columns`.

Previous chapter
< 17. Working with Large Databases

Next chapter
A. ER Diagram for Example Database >